HRCIM-NTT: Compute-in-Memory NTT Accelerator with Hybrid-Redundant Numbers

Paper study and Cadence Virtuoso implementation of one Butterfly Unit in Memory (BUIM)

Daksh Desai (24110098) Mitwa Goswami (24110205)

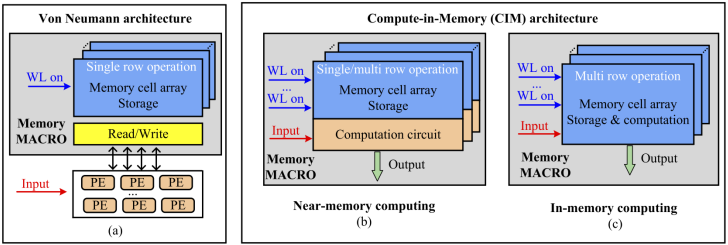
VLSI Design Course Project

October 2026

Paper: X. Zhang, Y. Wei, M. Li, J. Tian, Z. Wang, "HRCIM-NTT: An Efficient Compute-in-Memory NTT Accelerator With Hybrid-Redundant Numbers," IEEE TCAS-I, vol. 72, no. 1, pp. 214–226, Jan. 2025.

1. Motivation: why NTT, why compute-in-memory?

- Quantum computers (Shor) break RSA/ECC ⇒ NIST selected **post-quantum cryptography (PQC)**: Kyber, Dilithium, Falcon (lattice), SPHINCS+.
- Lattice schemes spend most time on **polynomial multiplication**. The **number-theoretic transform (NTT)** cuts it from $O(N^2)$ to $O(N \log N)$, but it is still the bottleneck: costly **modular multiplication** plus heavy **data transfer**.
- **Memory wall:** moving data between memory and processing elements (PEs) costs about 1000× the time and 40× the power of the arithmetic itself.
- **Compute-in-memory (CIM):** do the computation inside or right next to the memory array, so the transfers disappear.



Source: paper, Fig. 2: von Neumann vs. near-memory (NMC) and in-memory (IMC) computing

2. Introduction

Limits of prior in-SRAM NTT

- **Bit-serial** (MeNTT, Neural Cache): huge parallelism but high latency (modular multiply $> (L{+}1)^2$ cycles) and a transposed data layout.
- **Bit-parallel** (BP-NTT): simple layout but **carry propagation**; modular multiply takes $62L{-}8$ cycles.
- Both: shift-and-add gives L partial products; polynomial coefficients are rewritten into SRAM every round; butterflies run serially.

HRCIM-NTT contributions

1. **CFMM:** carry-free modular multiplication = radix-4 Booth + hybrid-redundant addition + on-the-fly reduction, only $L/2{+}2$ cycles.
2. **Data pattern:** only the constant **twiddle factors** are stored in SRAM.
3. **Fixed inter-stage routing** + one-stage pipeline: multiply and add/sub overlap.
4. Paper's result (45 nm CMOS): 0.18 μs latency, 16.73 nJ per NTT, 714 MHz.

3. Background: NTT and the butterfly unit

- Ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, prime $q \equiv 1 \bmod n$.
- $A_k = \sum_{n=0}^{N-1} a_n \omega_N^{nk} \bmod q$. INTT uses ω^{-1} and scales by N^{-1}.
- Cooley–Tukey NTT: $\log_2 N$ stages, $N/2$ **butterflies** per stage.
- One butterfly: $Y + X \cdot W$ and $Y - X \cdot W \pmod q$, where W is the **twiddle factor** (a constant root of unity).
- Kyber target in the paper: $N = 256$, $q = 3329 = 13 \cdot 2^8 + 1$, $L = 12$ bits.
- The expensive part of a butterfly is the **modular multiplication** $X \cdot W$. One BUIM computes exactly one butterfly.

Algorithm 1 Iterative CT-NTT Based on The Proposed BUIM

Input: $a(x) \in \mathbb{Z}_q[x]/(x^n + 1)$ in standard-order
Input: $\omega \in \mathbb{Z}_q[x]$ (precomputed table of 2n-th roots of unity, bit-reversed order)
Input: $n = 2^l$, q (s.t. $q \equiv 1 \bmod n$)
Output: $A(x) \in \mathbb{Z}_q[x]/(x^n + 1)$ in bit-reversed order

```
1:  t ← n/2
2:  for (m = 1; m < n; m = 2m) do
3:      k ← 0
4:      for (s = 0; s < m; s = s + 1) do
5:          W ← ω[m + s]
6:          for (j = k; j < k + 1; j = j + 1) do
7:              Y, X ← a[j], a[j + t]
8:              S ← CFMM(W, X)
9:              for (i = 0; i < M; i = i + 1) do
10:                 Z0[i] ← mr4a(Y[i], S[i])
11:                 Z1[i] ← mr4s(Y[i], S[i])
12:             end for
13:             a[j], a[j + t] ← Z0, Z1
14:         end for
15:         k ← k + 2t
16:     end for
17:     t ← t/2
18: end for
```

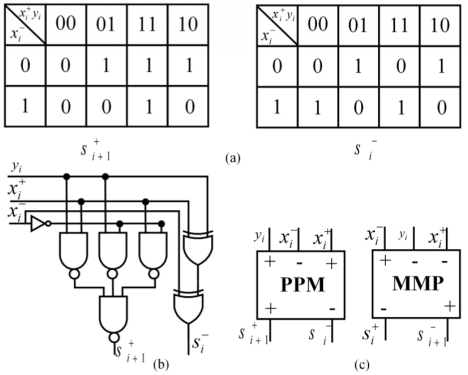
Source: paper, Algorithm 1

4. Redundant numbers and the PPM cell

- **Redundant signed digit:** $x_i = x_i^+ - x_i^- \in \{-1, 0, 1\}$. Many encodings per number, which allows **carry-free** addition.
- **Hybrid-redundant addition** $S = X + Y$: X redundant, Y in 2's complement. Per bit, $p_i = x_i^+ + y_i - x_i^- \in \{-1, 0, 1, 2\}$, rewritten as $p_i = 2t_i^+ - u_i^-$.
- **PPM (plus-plus-minus) cell** produces the next-digit bit s_{i+1}^+ and the current-digit bit s_i^- with no carry chain:

$$s_{i+1}^+ = x_i^+ y_i + \overline{x_i^-}(x_i^+ + y_i), \quad s_i^- = x_i^+ \oplus y_i \oplus x_i^-$$

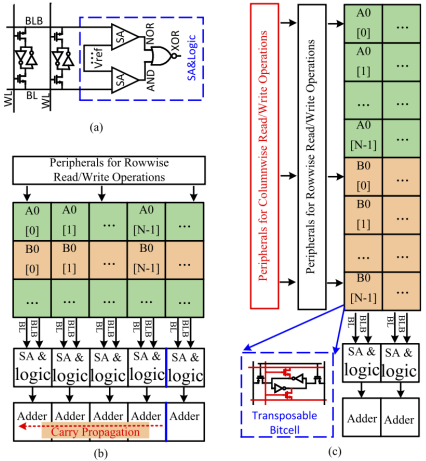
- Cost is about one full adder (FA), but the delay does **not** grow with word length. **MMP** (minus-minus-plus) is the same cell with different wiring.
- **Radix-4 digit** $\in \{-2, \ldots, 2\}$ is kept in 3 bits with weights $(-2, +1, +1)$. The **mrHY4A** cell (next slides) = FA + PPM.



Source: paper, Fig. 1: (a) Karnaugh maps, (b) gate-level PPM, (c) symbols

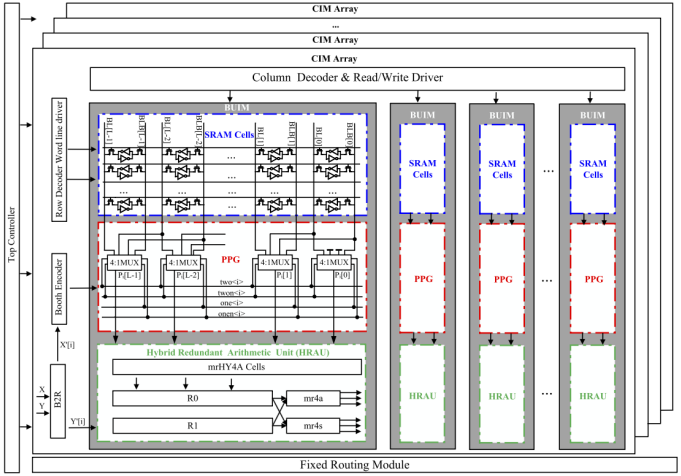
5. CIM basics: bitline logic in SRAM

- **Bitline computing (Fig. 3a):** precharge the bitline (BL) high and activate two wordlines (WL). If any cell holds 0, BL discharges: BL = AND, complementary line BLB = NOR, read by sense amplifiers (SA). XOR = NOR(AND, NOR).
- **NMC** (near-memory computing): ordinary SRAM plus an adjacent compute circuit. **IMC** (in-memory computing): the cells themselves compute.
- **Digital CIM (DCIM)** is exact; analog CIM is efficient but sensitive to PVT variation.
- **Bit-parallel (Fig. 3b):** operands stored row-wise, whole word at once, but **carries propagate** along the adders.
- **Bit-serial (Fig. 3c):** transposed data, one bit per cycle, high latency and complex peripherals.
- HRCIM-NTT: **bit-parallel multiplication + hybrid-redundant (carry-free) addition**.



Source: paper, Fig. 3: prior in-SRAM computing solutions

6. HRCIM-NTT overall architecture



Source: paper, Fig. 4: overall architecture

- Each **CIM array** holds many **BUIMs** (butterfly units in memory). BUIM = L SRAM columns + **PPG** (partial product generator) + **HRAU** (hybrid-redundant arithmetic unit).
- **B2R** converts binary inputs X, Y to redundant X', Y' (just wiring).
- The **Booth encoder** turns $X'[i]$ into select signals for the PPG.
- All twiddles of one NTT stage sit in **one SRAM row**; one WL activation serves every butterfly of that stage.
- Multiply is **bit-parallel**; add/subtract is **digit-serial**.
- **Fixed routing module** feeds results to the next stage.

7. CFMM algorithm: Booth recoding and on-the-fly reduction

Booth recoding. $X'[i] = -2x[2i+1] + x[2i] + x[2i-1] \in \{-2, \ldots, 2\}$, with $M = \frac{L}{2} + 2$ digits, so there are about half as many partial products as shift-and-add.

LSD-first multiply-reduce. With $W' = W \cdot 4^{M-1} \bmod q$:

$$P_i = W'X'[i], \quad T = S_{i-1}[M-1{:}1] + P_i, \quad S_i = T + S_{i-1}[0] \cdot 4^{-1} \bmod q$$

The $\div 4$ right shift keeps the width fixed.

Cheap correction. For $q = \delta \cdot 2^e + 1$: $4^{-1} \equiv -\delta\, 2^{e-2}$, so the dropped digit is compensated by adding $S_{i-1}[0] \cdot (-\delta)$ at digit position $e/2 - 1$ (R in Alg. 2).

Bound. The sum stays in $(-4q, 4q)$ with no extra cycles.

Kyber: $q = 3329$, $\delta = 13$, $e = 8$, $L = 12 \Rightarrow M = 8$ cycles per multiplication.

The result stays in the lazy range $(-4q, 4q)$; the paper does not detail a final canonical reduction (to verify).

Algorithm 2 Proposed Carry-Free Modular Multiplication (CFMM) Algorithm

Input: L-bit $q = \delta \cdot 2^e + 1$
Input: L-bit W' (pre-computed)
Input: M-digit X' (mr4)
Output: $S = W \cdot X' \bmod q$ (mr4)

1: $S = 0$ ▷ Initialize intermediate sum
2: $l \leftarrow \frac{e}{2} - 1$
3: **for** $(i = 0;\ i < M;\ i{+}{+})$ **do**
4: $two, twon, one, onen \leftarrow$ **Booth_Encoder**$(X'[i])$
5: $P_i \leftarrow \mathrm{PPG}(two, twon, one, onen, W')$
6: **for** $(j = 0;\ j < M - 1;\ j{+}{+})$ **do**
7: $\{t_{j+1}^{++}, t_j^{2-}, t_j^{+}\} \leftarrow \mathrm{mrHY4A}(P_i[2j{+}1{:}2j],\ S[j{+}1])$
8: **end for**
9: $t_0^{++} \leftarrow P_i[L+1]$ ▷ Plus sign bit
10: **for** $(k = 0;\ k < l;\ k{+}{+})$ **do**
11: $S[k] \leftarrow T[k]$
12: **end for**
13: $R \leftarrow S[0] \cdot (-\delta)$
14: **for** $(k = l;\ k < M;\ k{+}{+})$ **do**
15: $p \leftarrow 2(k - l)$
16: $\{s_{k+1}^{++}, s_k^{2-}, s_k^{+}\} \leftarrow \mathrm{mrHY4A}(R[\mathrm{p{+}1{:}p}],\ T[\mathrm{k}])$
17: **end for**
18: $s_l^{++} \leftarrow R[msb]$
19: **end for**

Source: paper, Algorithm 2: CFMM

8. Partial product generator (PPG) inside the SRAM

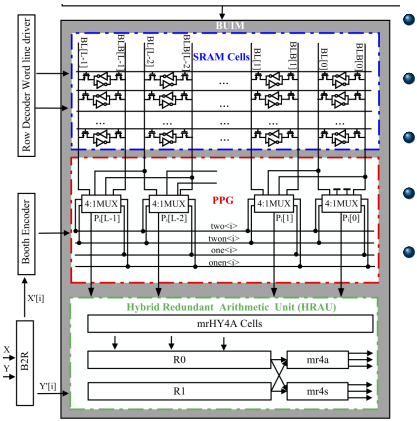
TABLE I

TRUTH TABLE OF BOOTH ENCODER

x $[2i+1]$	x $[2i]$	x $[2i-1]$	X' $[i]$	P_i	two $[i]$	$twon$ $[i]$	one $[i]$	$onen$ $[i]$
0	0	0	0	0	0	0	0	0
0	0	1	+1	w′	0	0	1	0
0	1	0	+1	w′	0	0	1	0
0	1	1	+2	2w′	1	0	0	0
1	0	0	-2	$\overline{2w'}$	0	1	0	0
1	0	1	-1	$\overline{w'}$	0	0	0	1
1	1	0	-1	$\overline{w'}$	0	0	0	1
1	1	1	0	0	0	0	0	0

Source: paper, Table I: truth table of the Booth encoder

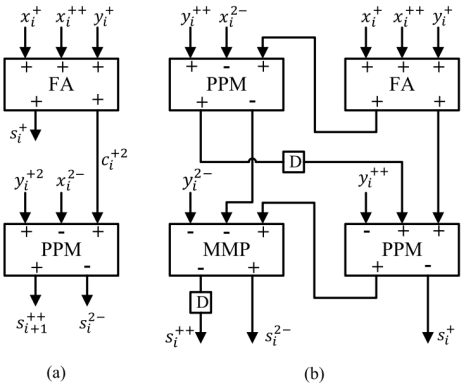
Encoder outputs (from Table I): one $= \overline{x_{2i+1}}(x_{2i} \oplus x_{2i-1})$, onen $= x_{2i+1}(x_{2i} \oplus x_{2i-1})$, two $= \overline{x_{2i+1}} x_{2i} x_{2i-1}$, twon $= x_{2i+1} \overline{x_{2i}}\ \overline{x_{2i-1}}$. Negation is a 1's complement; the +1 enters through the sign bit.



Source: paper, Fig. 4 (BUIM detail)

- **Twiddle W' sits in L SRAM cells of one row (2's complement).**
- **WL on $\Rightarrow$ BL carries W' and BLB carries W' (cross-coupled inverters give it for free).**
- **Shift $2W'$ = use the neighbouring bitline. Negate = use BLB.**
- **One 4:1 MUX per bit picks BL/BLB signals using one/onen/two/twon.**
- **About 25% of a normal Booth PPG's transistors; only 1.8% of total energy.**

9. Adder cells: mrHY4A and mr4a/mr4s



Source: paper, Fig. 6: (a) mrHY4A, (b) mr4a

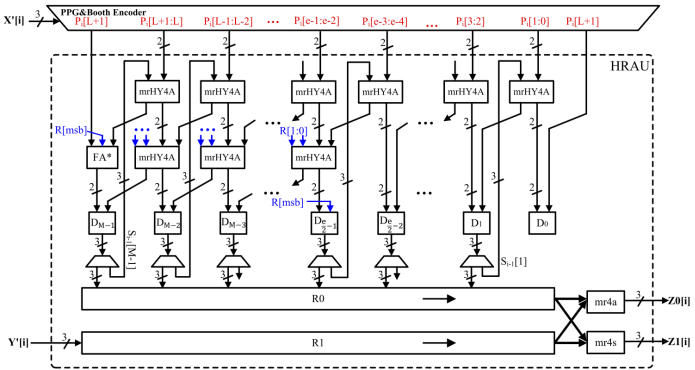
(a) mrHY4A (minimally redundant hybrid radix-4 adder): adds a redundant digit $(x^{\mathbf{2}-}, x^{+}, x^{++})$ to a 2-bit 2's-complement digit $(y^{+\mathbf{2}}, y^{+})$.

- **FA** adds the weight-1 bits $x^{+}, x^{++}, y^{+} \rightarrow$ sum s^{+} and carry $c^{+\mathbf{2}}$ (weight 2).
- **PPM** adds the weight-2 bits $y^{+\mathbf{2}}$, $c^{+\mathbf{2}}$ and subtracts $x^{\mathbf{2}-} \rightarrow s_{i+\mathbf{1}}^{++}$, $s_i^{\mathbf{2}-}$.
- Used in the HRAU: partial products accumulate with a delay of one cell.

(b) mr4a / mr4s (minimally redundant radix-4 adder / subtractor): both operands are redundant, so it is built from FAs, PPM, MMP and flip-flops (D) that hold the transfer digit between digit times.

- **mr4s** = mr4a with some wires re-connected, so subtraction needs **no 2's-complement step**.
- One digit per clock $\Rightarrow L/2 + 2$ cycles, equal to the multiplication time, so the pipeline never stalls.

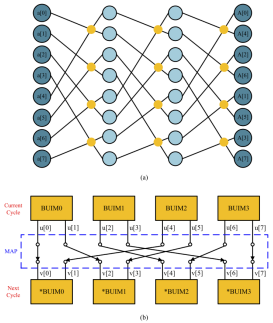
10. HRAU: accumulation and reduction without carries



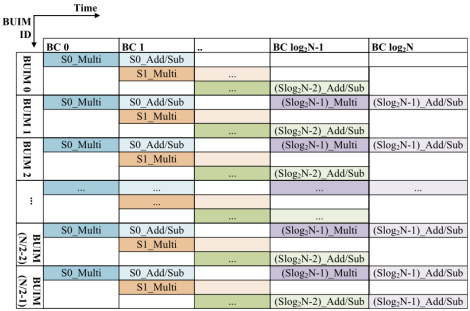
Source: paper, Fig. 5: hybrid-redundant arithmetic unit (HRAU)

- Each clock, one partial product P_i from the PPG enters a row of **mrHY4A** cells and is added to the previous sum **in all digits in parallel**.
- Registers $D_0..D_{M-1}$ hold the running sum S (3 bits per digit). The $\div 4$ shift is only **wiring**.
- D_0 has no outgoing wire (LSD abandoned). A **second-level** mrHY4A row adds $R = S[0] \cdot (-\delta)$ to the upper digits as compensation.
- When the multiply finishes, S is loaded into shift register **R0**; Y' sits in **R1**. The **mr4a/mr4s** pair outputs $Z0 = Y' + S$ and $Z1 = Y' - S$.
- Critical path: one mrHY4A, 1.33 ns in the paper, independent of L.

11. Dataflow: fixed routing and lightweight pipeline



Source: paper, Fig. 7: data mapping, 8-point NTT



Source: paper, Fig. 8: pipeline (BC = butterfly cycle)

- **Fixed routing:** $v[2i] \leftarrow u[i]$, $v[2i{+}1] \leftarrow u[i + N/2]$. The wiring is identical for every stage, so there is no crossbar and no remapping through memory.
- One **BC** $= L/2 + 2$ clocks. In each BC, add/sub of stage s and multiplication of stage $s{+}1$ run **simultaneously**. Total clocks per NTT $= (\log_2 N + 1)(L/2 + 2)$: $9 \times 8 = 72$ for N=256, L=12.

12. Results: cycle count and energy/area breakdown

TABLE II

COMPARISON ON CYCLES OF ARITHMETIC OPERATIONS PERFORMED BY DIFFERENT CIM-BASED SOLUTIONS

	Multiplication	Addition	Subtraction
This Work (HRCIM-NTT)	$\boldsymbol{L/2+2}$	$\boldsymbol{L/2+2}$	$\boldsymbol{L/2+2}$
MeNTT [21]	$(L+1)^2$	$2(L+1)$	$3(L+1)$
BP-NTT [22]	$62L-8$	$8L-8$	$8L-5$
Neural Cache [24]	L^2+5L-2	$L+1$	/
CryptoPIM [50]	$6.5L^2-11.5L+3$	$6L+1$	$7L+2$

Source: paper, Table II: cycles per operation

- Multiply takes about 1% of BP-NTT's cycles; the pipeline saves about 50% of NTT cycles.
- Our cross-check: $72 \times 1.4\,\text{ns} \approx 101\,\text{ns}$, not the reported $0.18\,\mu\text{s}$; and $1/0.18\,\mu\text{s} \approx 5.6\,\text{M}$ NTT/s, not the printed 5.58 K.

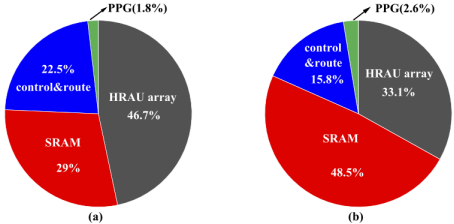


Fig. 9. Energy and area breakdown of HRCIM-NTT. (a) Energy. (b) Area.

Source: paper, Fig. 9: (a) energy, (b) area breakdown

HRAU is the compute-heavy part (46.7% energy, 33.1% area); SRAM takes 48.5% of area but under 30% of energy; PPG only 1.8% of energy.

13. Comparison with state of the art (256-point NTT)

TABLE III

COMPARISON TABLE WITH STATE-OF-THE-ART NTT ACCELERATORS ON A 256-POINT POLYNOMIAL

	This work	MeNTT [21]	BP-NTT [22]	RM-NTT [39]	CryptoPIM [50]	LEIA [54]	VPQC [55]	SA_NTT [56]	KyberMat [57]	CPU [50]
Method	**In-SRAM**	In-SRAM	In-SRAM	In-RRAM	In-RRAM	ASIC	ASIC	FPGA	FPGA	software
Technology	**45nm**	65nm	45nm	28nm	45nm	40nm	28nm	-	-	-
Bitwidth	**12**	14	16	14	16	14	12	16	12	16
Frequency (MHz)	**714**	151	3.8K	400	909	300	300	184	222	2K
Latency (μs)	**0.18**	23	61.9	0.28	68.7	0.53	0.137	21.6	0.67	85
Throughput (KNTT/s)	**5.58K**	42	258.6	3.57K	553.3	1.88K	7.3K	46	4.26K	11.8
Energy (nJ/NTT)	**16.73**	144	34	145	2.6K	31	21.4	21.5k	-	570K
†Normalized Energy (nJ/NTT)	**1.4**	6.5	2.1	17.6	162.5	2.5	3	-	-	-
Area ($\mu m^2 \times 10^3$)	**123**	360	63	*101	*152	*914	-	-	-	-
†Normalized Area ($\mu m^2 \times 10^3$)	**10.3**	12.6	3.9	*20.6	*9.5	*87	-	-	-	-

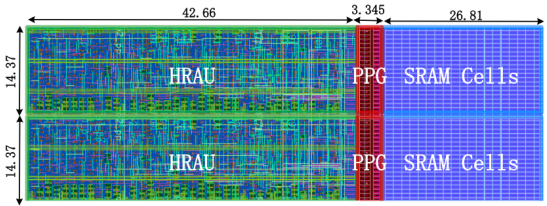
† The Energy and Area are normalized to 1 bit in the 45nm process.

* These works do not provide area overhead explicitly. We estimate their area by referring to work [22].

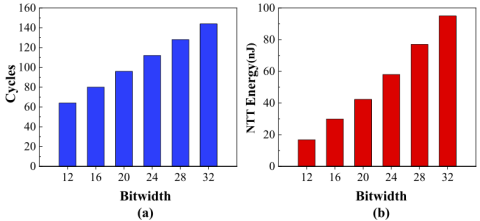
Source: paper, Table III

Compared with other in-SRAM designs (MeNTT, BP-NTT) HRCIM-NTT has far lower latency and lower normalised energy; BP-NTT is smaller in area. Latency and energy are the paper's reported values.

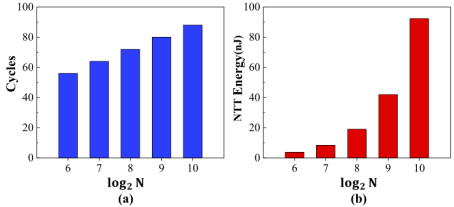
14. Layout and scaling plots



Source: paper, Fig. 10: layout of two adjacent BUIMs



Source: paper, Fig. 11: cycles and energy vs. bit width (N=256)



Source: paper, Fig. 12: cycles and energy vs. number of points (L=12)

- **Fig. 10:** each BUIM is HRAU (42.66 μm) + PPG (3.35 μm) + SRAM (26.81 μm), 14.37 μm tall; the HRAU is folded to match the SRAM pitch.
- **Fig. 11:** cycles grow linearly with bit width, energy roughly quadratically.
- **Fig. 12:** cycles grow with $\log_2 N + 1$; energy rises sharply as the array grows in both directions.
- The paper shows no transient waveforms; Fig. 8 is its timing diagram.

15. Our project: one BUIM in Cadence Virtuoso

Scope: one BUIM with the paper's parameters: $L = 12$, $q = 3329$ (δ=13, e=8), $M = 8$.

Approach

1. Write **Verilog** for each unit: Booth encoder, PPG mux, PPM/MMP/FA, mrHY4A, HRAU, mr4a/mr4s, registers.
2. See **which gates** each unit uses.
3. **Recreate those gates in Cadence Virtuoso** as schematics.
4. Turn each unit into a **symbol** and **reuse** it one level up: cells $\rightarrow$ mrHY4A $\rightarrow$ HRAU $\rightarrow$ BUIM.

Verification: test **one BUIM unit** (one butterfly, $Y \pm X \cdot W \bmod q$).

Waveforms we will show

1. SRAM read: WL, BL/BLB.
2. Booth encoder: $x[2i{+}1..2i{-}1] \rightarrow$ one/onen/two/twon.
3. PPG output P_i for each of the 8 digits.
4. HRAU registers D_k over 8 clocks; final S in R0.
5. mr4a/mr4s outputs $Z0, Z1$ over the next 8 clocks.
6. PPM/mrHY4A truth-table check.

To check: final reduction of the $(-4q, 4q)$ result is not detailed in the paper.

16. References

References

1. Zhang et al., HRCIM-NTT, IEEE TCAS-I, 72(1), 2025.
2. Li et al., MeNTT, IEEE TVLSI, 30(5), 2022.
3. Zhang, Imani, Sadredini, BP-NTT, arXiv:2303.00173.
4. Eckert et al., Neural Cache, ISCA 2018.
5. Parhi, *VLSI Digital Signal Processing Systems*, Wiley, 2007.
6. Longa, Naehrig, NTT for lattice crypto, CANS 2016.